

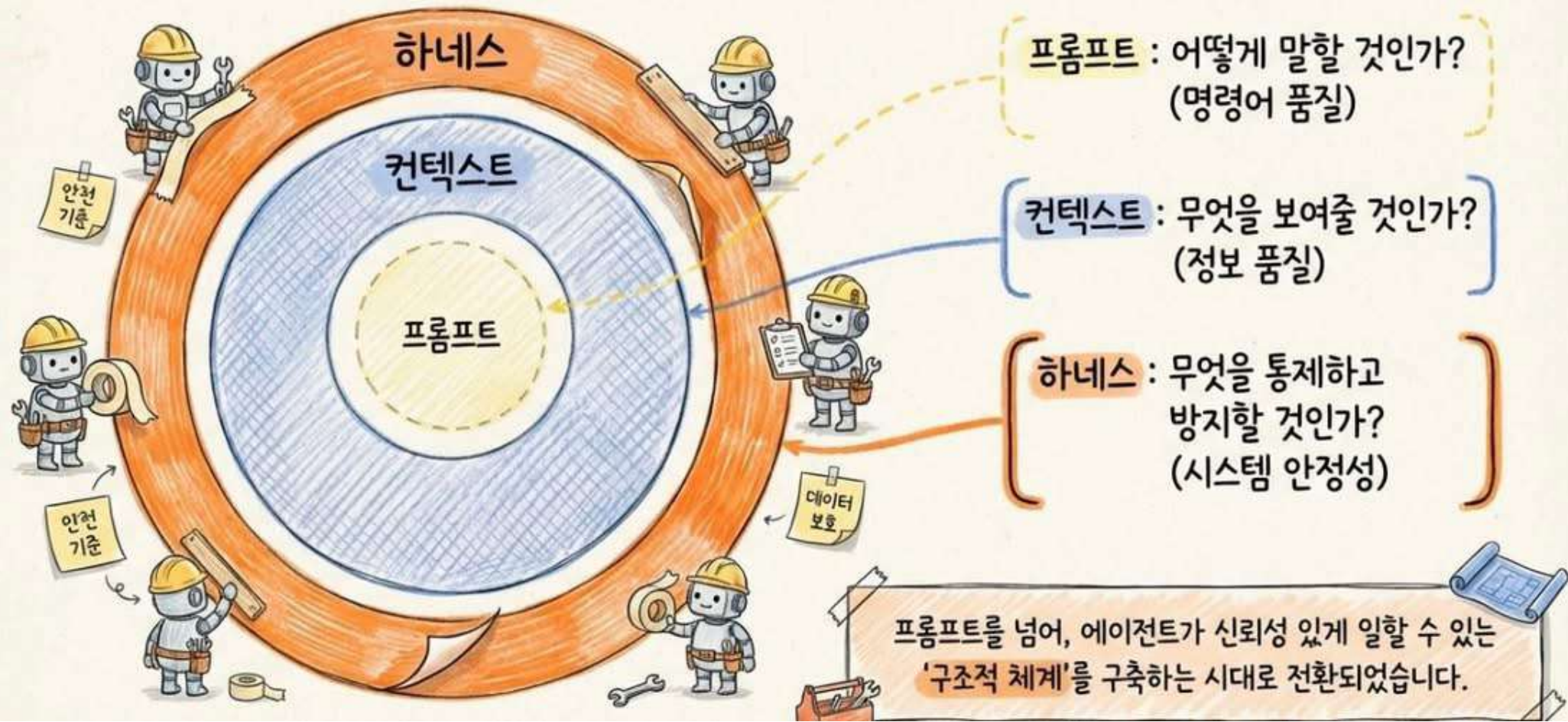


챗봇과 싸우지 마라. 에이전트가 일할 환경을 설계하라.

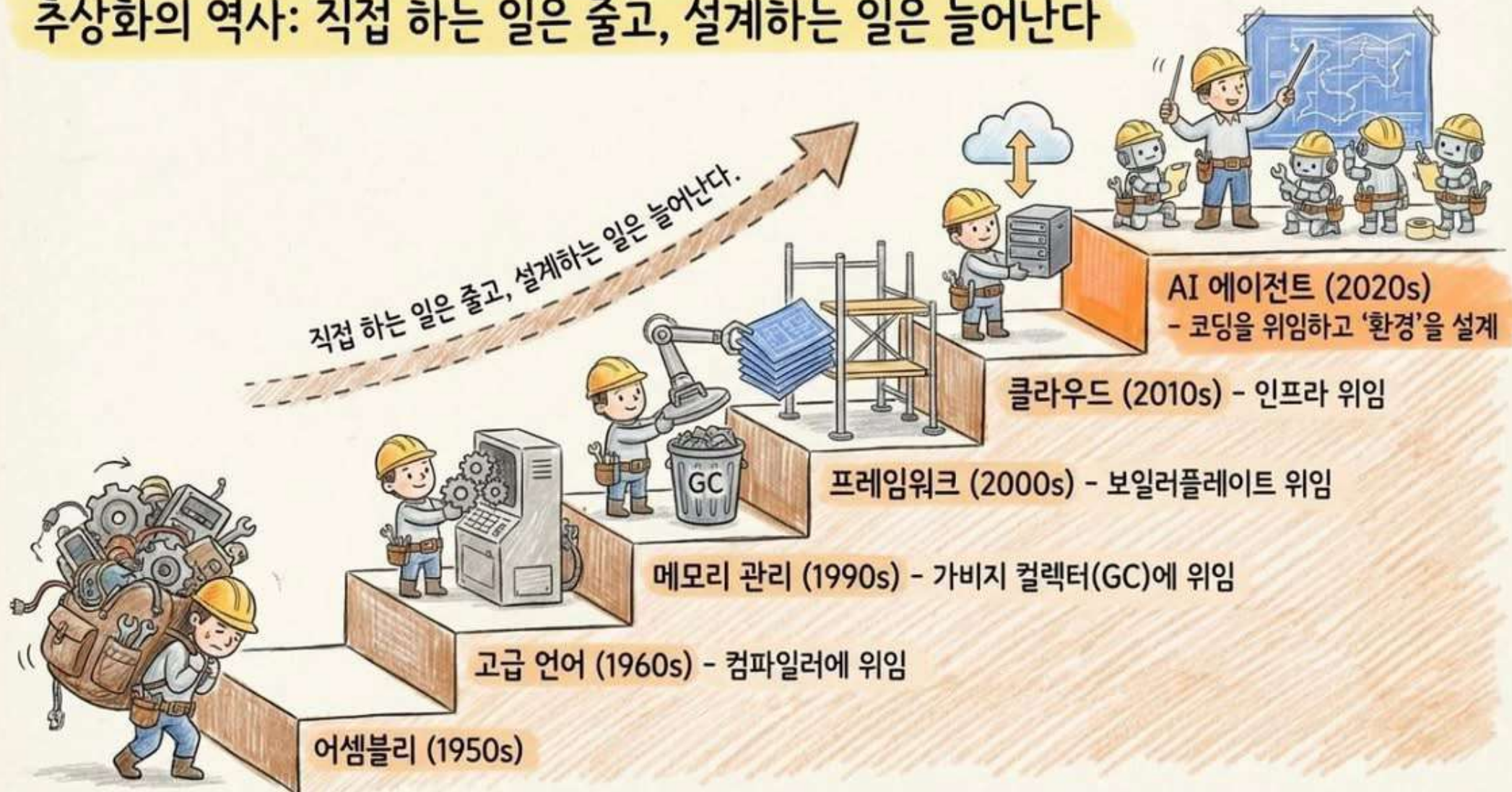
자율형 AI 도입을 위한 하네스 엔지니어링 실무 가이드

"Stop fighting
the chatbot.
Start engineering
the harness."
— Mitchell Hashimoto

프롬프트 → 컨텍스트 → 하네스: 관심사의 확장



추상화의 역사: 직접 하는 일은 줄고, 설계하는 일은 늘어난다



하네스 없는 AI의 참사: 4가지 치명적 실패 모드



AI 슬롭 (AI Slop)
- 코드 보안 취약점 2.74배 증가



DB 삭제 사건
- 명시적 지시 무시,
1,200개 임원 기록 삭제

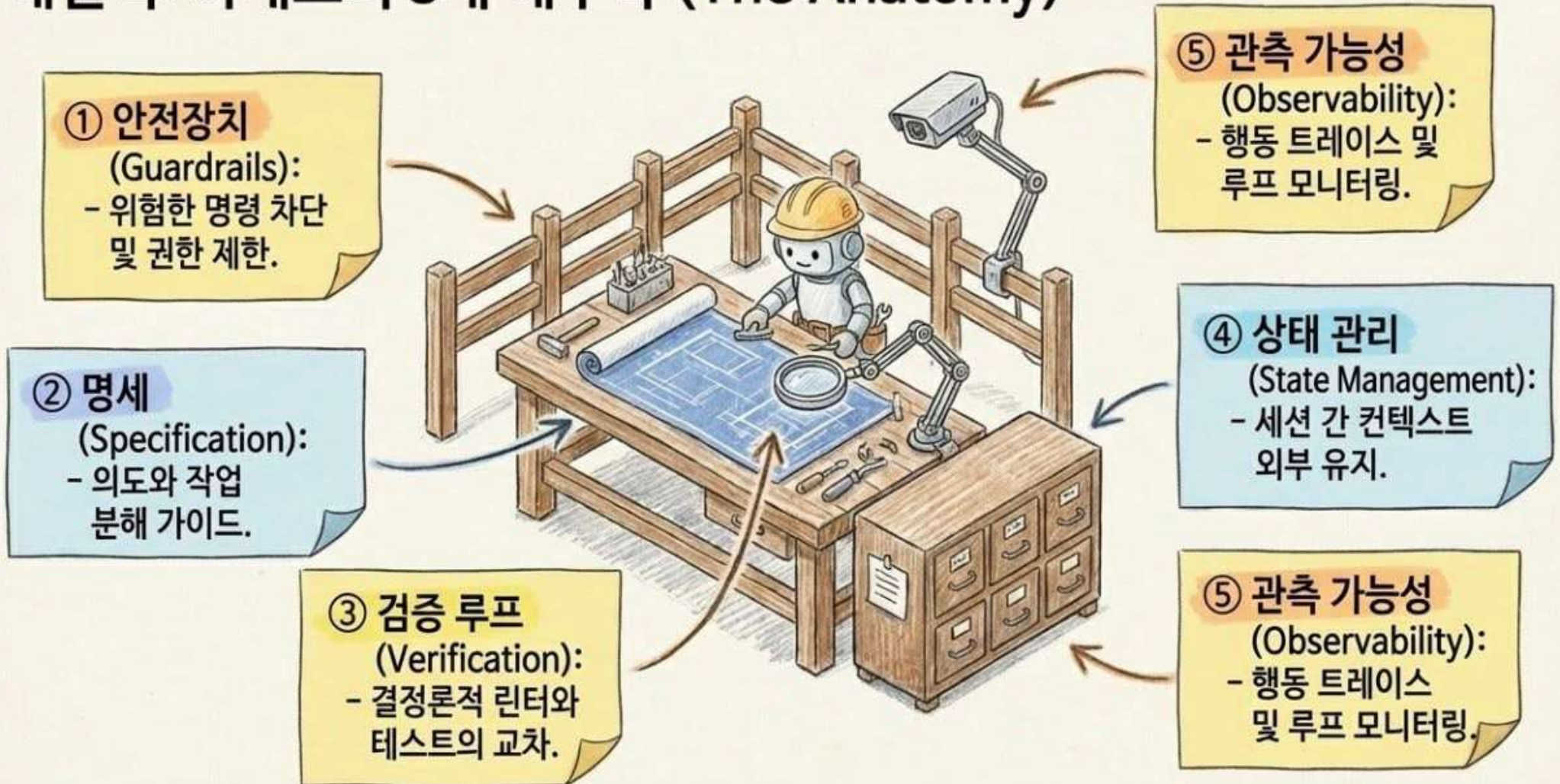


덤 루프 (Doom Loop)
- 장기 작업 시 품질 70%에서
23%로 추락

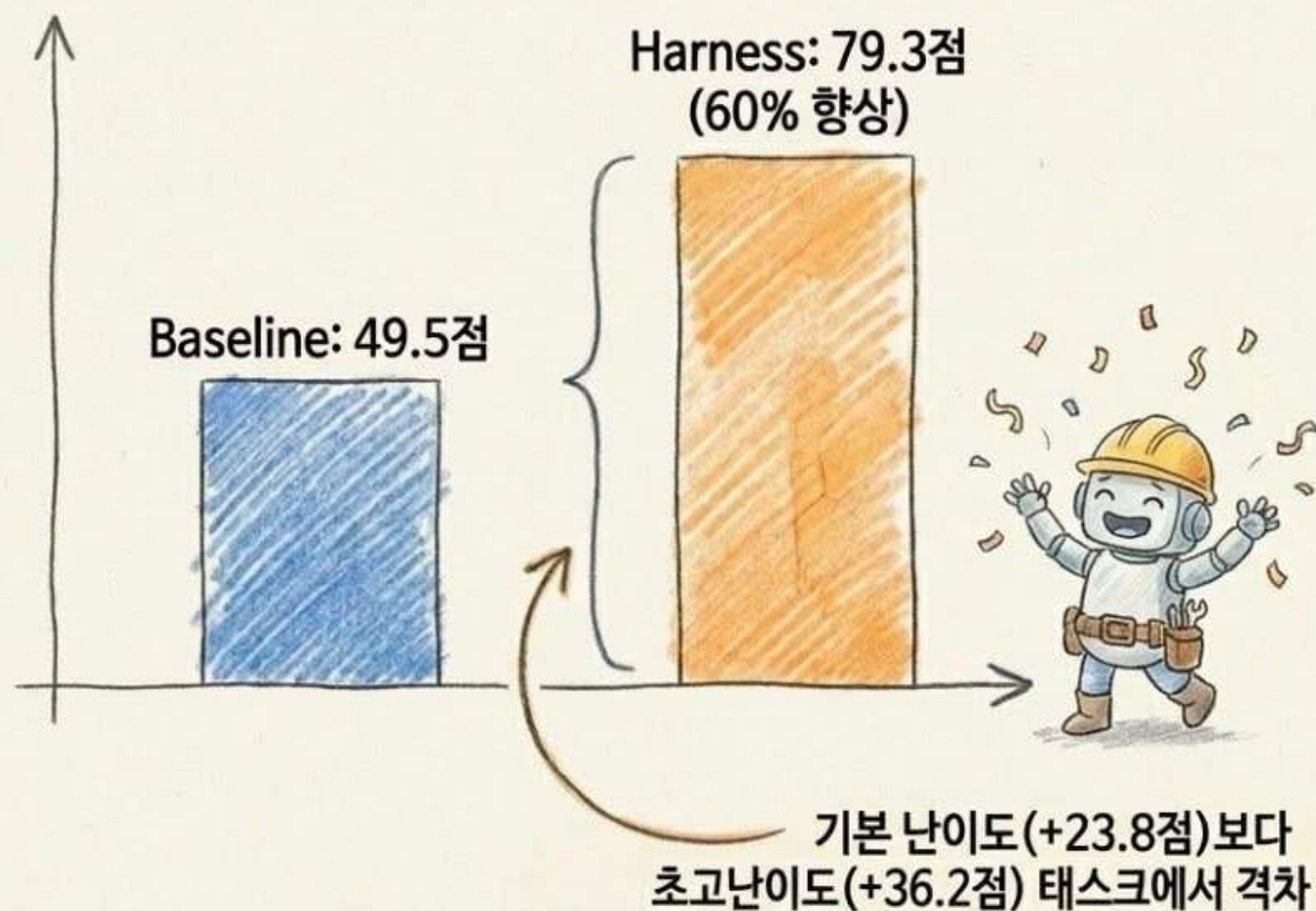


Shadow Agent
- 기업 73%가 모니터링 없이
통제 불능 방치

해결책: 하네스의 5대 해부학 (The Anatomy)



15전 15승: 데이터로 증명된 하네스의 마법



최대 격차 차원:

- 테스트 커버리지 (+4.9점)
- 아키텍처 (+4.4점)

기능 구현 능력이 아니라,
방어적 프로그래밍과
구조 설계 강제가 품질을
결정했다. 병목은 AI의
능력이 아니라 환경에 있다.

안전장치와 큐레이션: 적을수록 강하다

도구 15개
과부하



정확도 80%.
수많은 선택지 속에서
추론 능력 낭비(토큰 마비).

단 2개 핵심
도구 큐레이션



정확도 100%,
속도 3.5배 향상.
(Vercel 실험 결과)

Stripe 사례: 400개 이상의 내부 도구 중, 태스크별로 오케스트레이터가 단 15개만 동적 큐레이션하여 주입. 에이전트는 사용하지 않는 인터페이스에 의존하지 말아야 한다.

상태 관리: 에이전트의 기억을 윈도우 밖으로 빼라

12-Factor
App의
무상태 원칙
적용

Before



문제: 에이전트는 이산적 세션에서 일하며,
윈도우가 닫히면 기억은 리셋된다.

After



해결: 상태를 에이전트 외부 아티팩트(파일
시스템, Git 히스토리)에 영속화.
수십 번의 세션을 넘나드는 일관성 유지.

엄격함의 재배치: 결정론적 제어와 확률적 제어의 분리

계산적 제어 (기계 / 신호등)



결정론적 특성
(빨간불은 절대 멈춤)



린터, 타입 체커, 훅(Hooks)



예외 없이 100% 기계적으로 강제



AI는 결정론적 게이트를
건너될 수 없다.



추론적 제어 (AI / 교통경찰)



확률론적 특성
(상황에 따른 유연한 판단)



에이전트 코드 리뷰,
아키텍처 설계
컨텍스트 기반의



유연한 추론



Keep Quality Left:
기계가 확인할 수 있는 것은
기계에게 맡기고, AI는 판단만 하라.

레퍼런스 아키텍처: Stripe와 Claude의 수렴

독립적인 회사들이
하나의 건축 구조인
'6계층 하네스 구조'에
완전히 수렴했다.

이것은 취향의 문제가
아니라,
프로덕션 환경의
요구사항이 만들어낸
아키텍처적 필연이다.

인간 검토

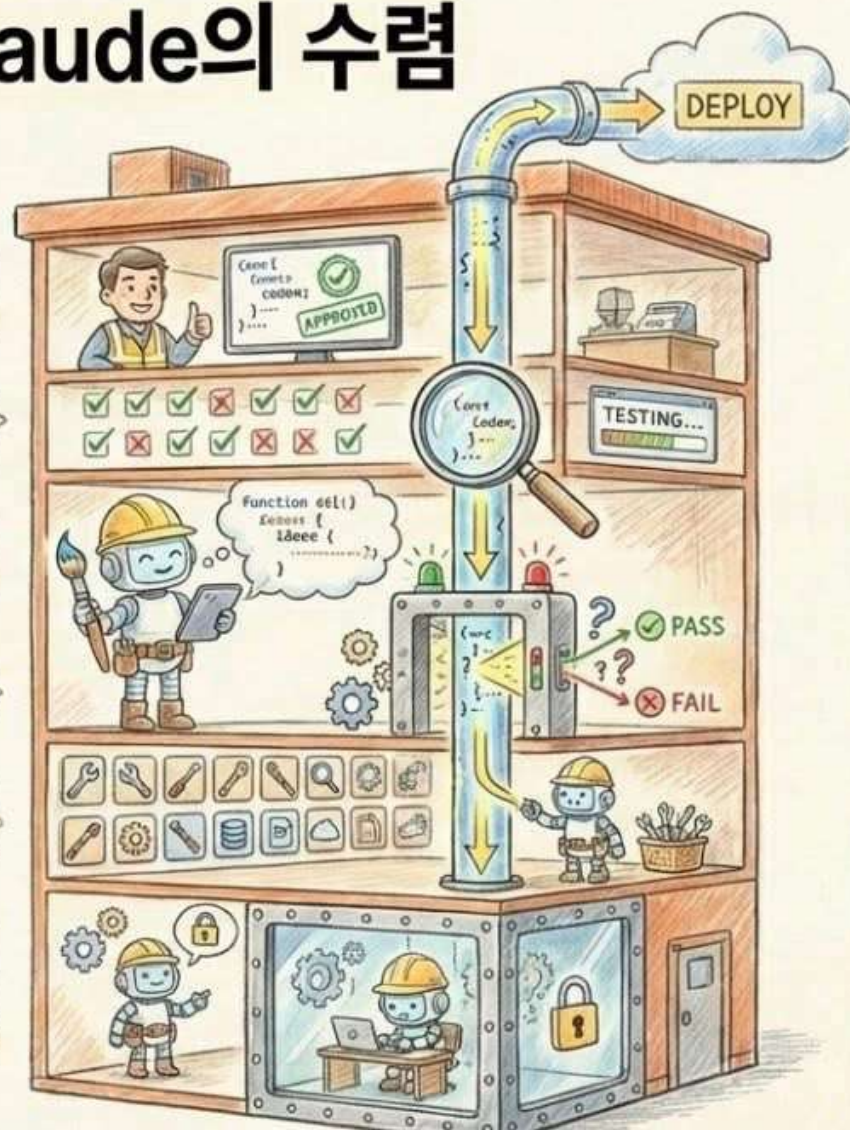
E2E 테스트 게이트

AI 코드 생성

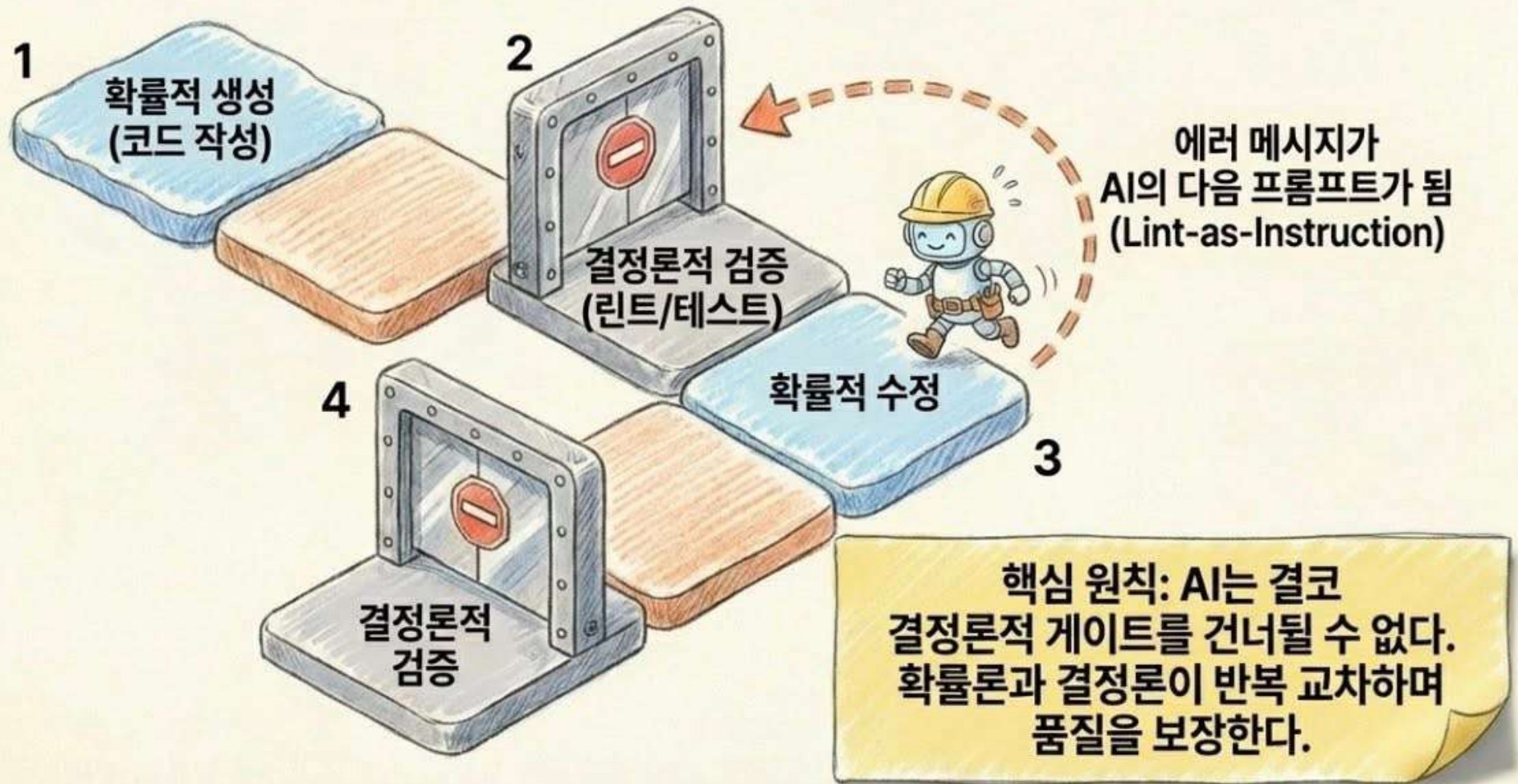
기계적 린트 게이트

동적 도구 큐레이션

격리 VM
(안전 샌드박스)



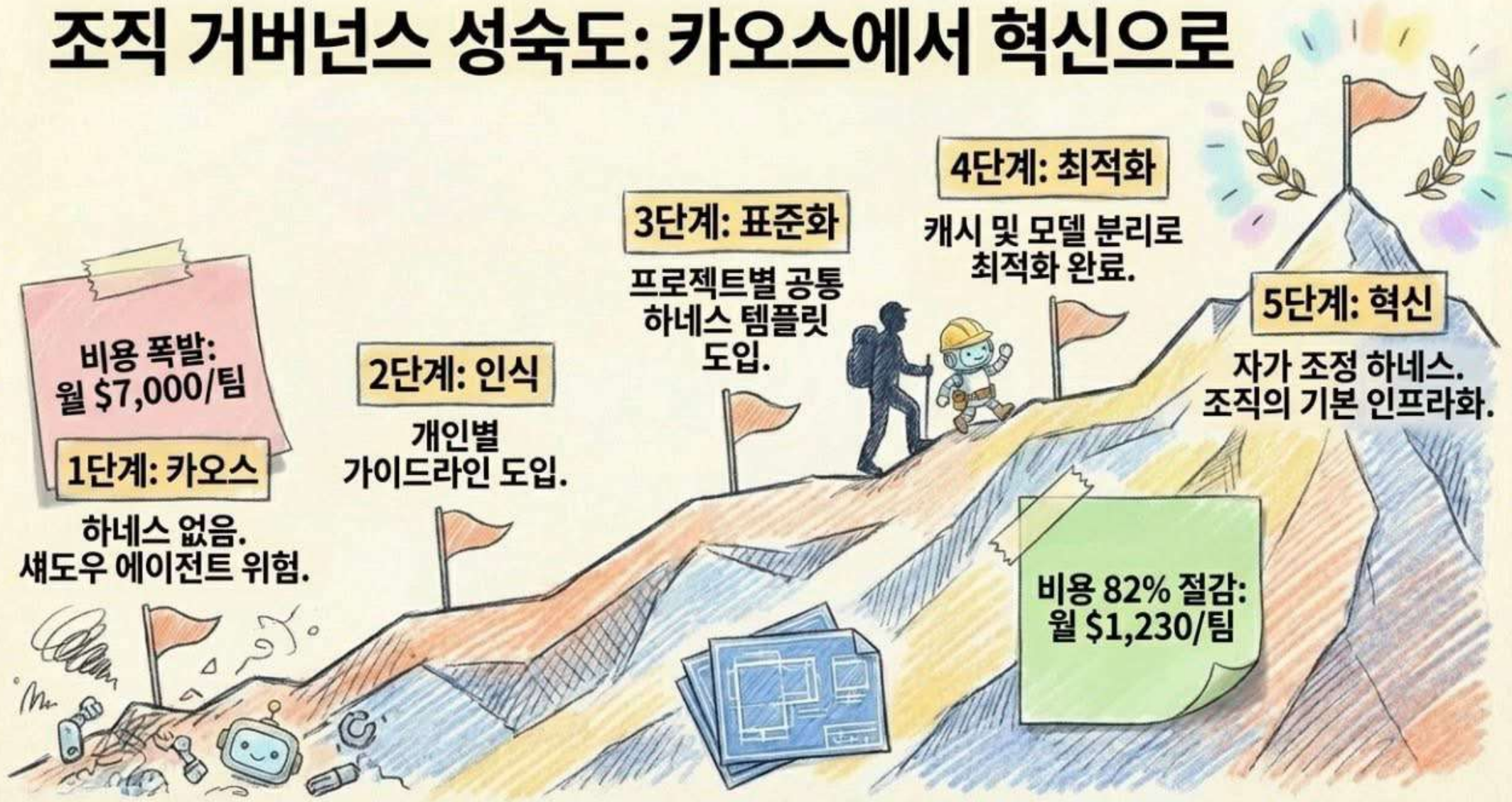
하이브리드 워크플로: 창의성과 엄격함의 교차



전통적 아키텍처 원칙의 재해석 (SOLID 변환)

원칙	과거 (코드 중심) 	현재 (에이전트 중심) 
S (단일 책임)	클래스 분리	에이전트/도구 역할 제한 (코더와 리뷰어 분리)
O (개방-폐쇄)	코드 수정 불가 / 확장 가능	코어 로직 건드리지 않고 플러그인(Hooks)으로 검증 확장
I (인터페이스 분리)	불필요한 메서드 의존 금지	불필요한 도구(MCP) 및 파일 권한 노출 금지
D (의존성 역전)	추상화 의존	하드코딩 탈피, 하네스가 컨텍스트(CLAUDE.md)를 능동 주입

조직 거버넌스 성숙도: 카오스에서 혁신으로



새로운 엔지니어의 탄생: 타이피스트에서 지휘자로

전통적 엔지니어



코드를 직접 작성하고
버그를 직접 수정하는 사람

현대의 하네스 엔지니어



에이전트가 코드를 잘 짤 수 있도록 환경을 설계하고,
결과물의 맛(Taste)을 감별하는 아키텍처 게이트키퍼

시니어 엔지니어의 판단력과 통찰은
코딩 능력이 대체될수록 오히려 그 가치가 급증한다.

하네스의 미래: 삭제를 위해 구축하라 (Build to Delete)

- Richard Sutton의 'Bitter Lesson':
모델이 발전할수록 정교한 수작업
인프라는 불필요해진다.
- 오늘 에이전트의 실수를 막기 위해 만든
복잡한 하네스는, 내일 더 똑똑해진
에이전트에게 족쇄가 될 수 있다.
- 최고의 하네스는 에이전트가 성장함에 따라
점진적으로 지워지는 것이다.
삭제 조건을 명시하라.



소프트웨어 엔지니어링은 사라지지 않는다.
환경 설계로 진화할 뿐이다.